Experiment 1

20/7/26

Study of various network commands

AIM:- Study of various network commands used in linux and windows

Commands.

1) arp -a :- ARP is short form of address resolution protocol, It stores IP address and MAC address of system.

Output :-

```
Interface : 172.16.11.144 --- 0xa
  Internet Address      Physical address      Type
  172.16.8.1            7c-5a-1c-cf-be-15     dynamic
  172.16.8.133          00-27-0e-13-f0-9e     dynamic
  .
  .
  .
  255.255.255.25        ff-ff-ff-ff-ff-ff     static
```

2) hostname : This is the simplest of ~~all~~ ~~TCP~~ IP. To display ~~current~~ name of computer.

Output : DEV.

3) ipconfig : Display TCP/IP connection including Router, Gateway, DNS, DHCP and type of Ethernet adapter in your system.

Output :-

```
Ethernet adapter Ethernet
   Connection-specific DNS suffix.
   IPv4 Address. . . . . : 172.16.11.144
   Subnet Mask . . . . . : 255.255.252.0
   Default Gateway . . . . . : 172.16.8.1

Wireless LAN adapter
   Media State . . . . . : Media disconnected
   Connection-specific DN
```

4) netstat : netstat displays a variety of statistics of active TCP/IP connections. It can also display routing tables, interface statistics etc

Output :

Active connections

Proto	Local Address	Foreign Address	State
TCP	127.0.0.1:1042	DEV:63701	Established
TCP	127.0.0.1:1042	DEV:63702	"
⋮			
TCP	172.16.11.194:50013	a-0003 https	Time wait

5) nslookup : Tool used to perform DNS lookups in Linux. It is used to display DNS details such as IP address.

Output :

Server : Unknown
Address : 172.16.8.1

Non-authoritative answer :
Name : google.com
Addresses : 2404:6800:4007:805::200e
142.250.67.46

6) pathping : It traces the route to destination address that launches a 25 second test.

Output :

Tracing route to google.com [142.250.77.110]
over max of 30 hops

0 DEV [172.16.11.194]
1 .
: .

7) ping: Connect to host correctly between two nodes

Output:

Pinging google.com [142.250.67.46] with 32 bytes of data.

Reply from 142.250.67.46: bytes=32 time=9ms TTL=118

Reply from 142.250.67.46: bytes=32 time=17ms TTL=118

Reply from 142.250.67.46: bytes=32 time=9ms TTL=118

Ping statistics for 142.250.67.46:

Packets: Sent=4, Received=4, Lost=0 (0% Loss),

Approximate round trip times in milli seconds:

Minimum = 9ms, Maximum = 24ms, Average = 17ms.

Imp linux cmds

8) ip: Shows address information, manipulate routing, plus display network various devices, interfaces & tunnels.

(a) ip address show - to show Ip address assigned to an interface on your server.

Output:

1) lo <LoopbackUP, LowerUp>

inet 127.0.0.1

inet 6::1/128

2) enp0s3: <Broadcast, multicast, up, LowerUP>

inet 10.0.2.15/24

inet 6 fd17:625c:f037:2:900:27ff:fefe:/64

inet 6 fc80::a00:27ff:fefe:38fd/64

(b) ip address add 192.168.1.254/24 dev

To assign an IP to an interface

c) ip address del 192.168.1.254/24 dev enp0s3. To delete an IP on an interface.

d) ip route add 192.168.1.0/24 via 192.168.1.254
Add a route 192.168.1.0/24 via the gateway at 192.168.1.254.

e) ip route add 192.168.1.0/24 dev enp0s3
Add a route to 192.168.1.0/24 that can be reached on device enp0s3.

2) ifconfig: Used for configuration and troubleshooting network.

Output:

enp0s3: flags: 4163 <UP, Broadcast, running multicast>

inet 10.0.2.15 netmask 255.255.255.0

inet 6 fe80::a00:27ff:fefe:331d

inet 6 fd17:62fe:f037:2:900:27ff:fefe

lo: flags: 75 <UP, Loopback, Running>

inet 127.0.0.1 netmask 255.0.0.0

inet 6 ::1 prefixlen 128

loop txqueuelen 1000

3) mtr: It's a program with CLI that serves as a network diagnostic and troubleshooting tool.

Output:

localhost.localdomain (10.0.2.15) → google.com

Host

1. fra01501.ie.fl4.1.0100.net.

ping : Ping is a tool that verifies IP-level connectivity to another TCP/IP computer by sending ICMP echo request messages.

Output

$ ping google.com.

PING google.com (216.58.206.174) bytes of data
64 bytes from soft02s27-in-f14.1e100.net (216.58.206.174): icmp_seq=1 ttl=57 time 10.20.

64 bytes



Result : Thus the Basic commands for networking executed successfully

30/7/24

AIM-2 Study of different types of network cables

Types of cables

(i) UTP
(ii) STP
(3) Coaxial cable
(iv) Fibre Optic cable

Category

i) UTP



Advantage

- Cheaper in cost
- Easy to install as they have a smaller overall diameter

Disadvantage

- More prone to (EMI) Electromagnetic interference and noise

Application use

10 Base T ethernet

Fast ethernet, Gigabit Ethernet

Fast ethernet, Gigabit ethernet

STP





Advantage

- Shielded
- Faster than UTP
- Less susceptible to noise and interference

Disadvantage

- Expensive
- Complex installation effort
- High bandwidth
- Immune to interference
- Low loss bandwidth
- Versatile

Coaxial cable



Advantages

- High bandwidth
- Immune to interference
- Low loss bandwidth
- Versatile

Disadvantage

- Limited distance
- Cost
- Size is bulky

Application

Speed of signal is 500m television network High speed internet connection

fibre optics



Advantage

- High speed
- High bandwidth
- High security
- Long distance

Application of use

- Maximum distance of fibre optics cable is around 100 miles

Disadvantage

Expensive

- Requires skilled installer

Result :- Thus, the study of different networking cable was completed successfully.

Experiment 3

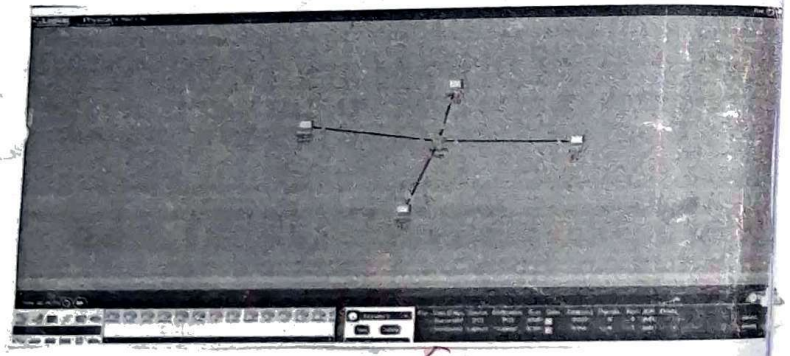
To study the packet tracer & analyze the behaviour of network devices

AIM: To understand environment of Cisco packet tracer. to design simple network.

Introduction

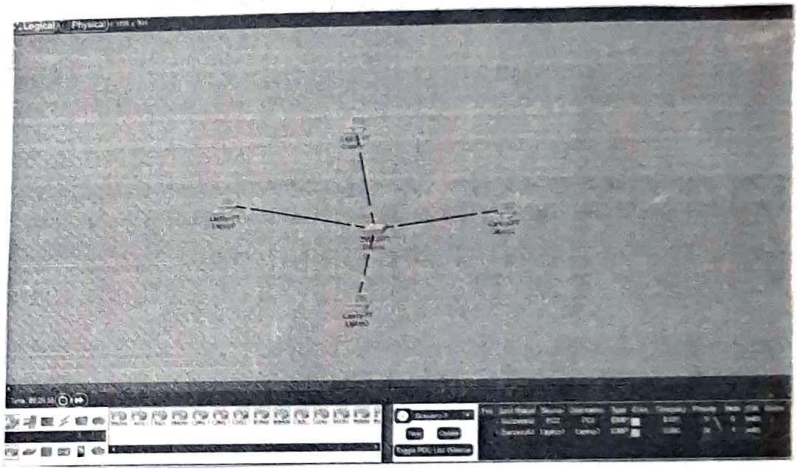
A simulation simulates network device and its environment. Packet tracer is an exciting network design simulation and modelling tool.

- It allows to model complex systems without the need for dedicated equipment.
- Protocols in packet tracer are coded to work and behave in the same way as they would on real hardware.



Observation 1: Hub

1) A hub forwards the received data to all connected devices without checking the destination.
2) Every PC connected to the hub receives the frame.
3) Only the destination PC accepts the frame other discard
4) It does not use mac addresses.
5) Network traffic is higher because data is broadcast to all ports. (Layer 1) OSI model.



Observation-2. Switch

1) A switch forwards data only to destination device
2) It learns mac address of connected devices.
3) Other PC's do not receive the frame
4) Switch works on data link (Layer 2) OSI model.
5) Network traffic reduced.

IP configuration Used

Device	IP Address
PC 0	10.0.0.1
PC 1	10.0.0.2
PC 2	10.0.0.3
PC 3	10.0.0.4

Network Topology

- Topology → Star
- Reason: All PCs are connected to a central hub or switch.

Result: Thus, the Cisco packet tracer has been used to simulate hub and switch successfully.

Experiment 1

Setting up a local area network (LAN)

AIM:- To design, set up and configure a LAN connecting four computers through a network switch, assign static IP addresses to all devices, verify connectivity using the ping command.

Equipment Requirement

S.No	Equipment	Quantity	Specification
1	Personal Computers	4	Ethernet (NIC) enabled
2	Network switch	1	8/16/24 port unmanaged/managed
3	Ethernet cables (Cat 5e / Cat 6)	4	Straight-through, RJ-45 connectors

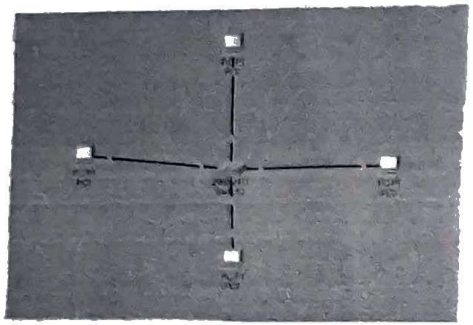
Theory:

A LAN is a network that connects computers and devices within a limited geographical area such as a home, office or laboratory, allowing them to share resources such as files, printers and internet connections.

A star topology is used in the experiment. The switch operates at layer 2 of the OSI model and forwards data frames between connected devices based on their MAC address.

Network Diagram:

The LAN was implemented using a star topology as shown below, with PCs connected individually to a central switch.



Implementation in CISO packet tracer

Step 1: Place the devices

1) Open cisco packet tracer.
2) From the device category panel, select End devices.
3) Select switch and drag switch.

Step 2: Cable the Devices

4) Click connections → select copper straight through cable.
5) Click each PC, the click the switch and select an available Fast Ethernet port to complete each connection.

Step 3: Assign IP addresses to PCs

7) Click a PC → Desktop → IP configuration
8) Enter the static IP address and subnet mask as per the IP table for each PC.

Step 4: Verify connectivity using ping

9) Click a PC → Desktop tab → Command prompt
10) Use the ping command to test connectivity to other PC eg ping 10.1.1.2

Step 5: Demonstrate Data sharing

11) Open the server's services (config) tab and enable the HTTP service.
12) From another PC, open the web browser (desktop tab) and enter the server's IP address to load the page, confirming successful data transfer.

Result: Thus, the LAN connecting switch and data transfer has been demonstrated successfully.

Experiment 5

Experiment on Packet capture tool: Wireshark

AIM: To perform experiments on packet capture tool wireshark.

Capturing and analysing packets using wireshark tool: To filter, capture, view packets in wireshark tool.

Procedure:

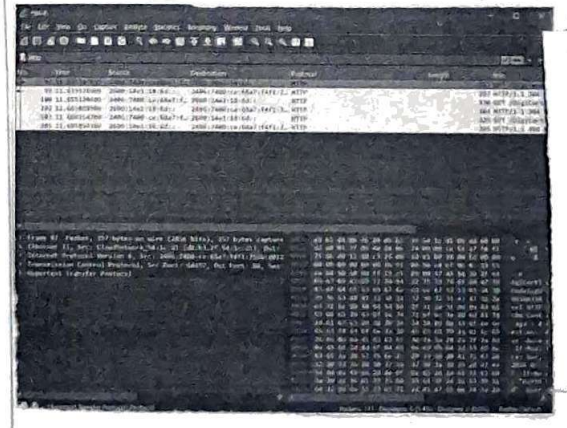
1) Select LAC in wireshark.
2) Go to capture option.
3) Select stop capture automatically after 100 packets.
4) Then, click start capture
5) Save the packets.

1) Create a filter to display only TCP/UDP packets inspect the packets and provide the flow graph.
   → Select Local Area connection in Wireshark.
   → Go to capture option.
   → Select stop capture automatically after 100 packets.
   → Then click start capture.
   → Search TCP packets in search bar
   → Save the packets.

2) Create a filter to display only ARP packets and inspect the packets.
   → Go to capture option.
   → Select stop capture automatically after 100 packets.
   → Then click start capture
   → Search ARP packets in search bar
   → Save the packets.

3) Create a filter to display only HTTP packets.
   → Select Local area connection in wireshark
   → Go to capture.
   → Click start capture.
   → Search HTTP packets in search bar.

4) Create a filter to display only IP/ICMP
   → Select Local area connection in wireshark.
   → Go to capture option.
   → Start capture
   → Search ICMP/IP packets in search bar.



Result: Thus. TCP/UDP ARP HTTP, IP/ICMP packet was executed.

Experiment 6

Hamming code

AIM: To write a program to implement error detection and correcting using hamming code and to test raw data input stream and verify error correction feature.

Code:-

```
data = input ("Enter data bits")
n = len (data
r = 0
while 2**r < n+r+1:
    r+=1
print ("Redudancy bits:", r)
total = n+r
h=[0]*(total+1)
j=0
for i in range (total, 0, -1):
    if (i*(i-1)) != 0:
        h[i] = int (data[j])
        j+=1
for p in range (r):
    pos = 2**p
    parity = 0
    for i in range (1, total+1):
        if i & pos:
            parity = parity ^ h[i]
    h[pos] = parity
print ("Data to be transmitted:", end=" ")
for i in range (total, 0, -1):
    print (h[i], end=" ")
print ()
change = input ("Is there any change in transmitted (Yes/No:")
if change.lower() == "yes":
    received = input ("Enter received data:")
    j=0
    for i in range (total, 0, -1):
        h[i] = int (received[j])
        j+=1
    error_position=0
    for p in range (1, total+1)
    if i & pos:
        parity = parity ^ h[i]
    if parity:
        error_position += pos
print ("Error position:", error_position)
```

Output:

① Enter data bits: 1101110
Redudancy bits: 4
Data to be transmitted: 1100111100
Is there any change is transmitted int? (Yes/No): Yes
Enter received data: 1100101001
Error position: 5

② Enter data bits: 1011011
Redudancy bits: 4
Data to be transmitted: 10101010111

Result: Thus, the Hamming code was implemented successfully for error detection and correction.

Experiment 7

Sliding window

AIM: To write a program to implement flow control at data link layer using sliding window protocol and to simulate the flow of frames from the node to another.

Code:-

sender.py

```
import time
window_size = int(input("Enter window size:"))
message = input("Enter the message:")
frames = []
for i in range(0, len(message), 5):
    data = message[i:i+5]
    frames.append([i//5, data])
print("\nFrame created:")
for frame in frames:
    print("Frame No", frame[0], "Data:", frame[1])

start = 0
while start < len(frame):
    end = min(start + window_size, len(frames))
    print("\nSending frames...")
    with open("Sender_Buffer.txt", "w") as file:
        for i in range(start, end):
            file.write(f"{frame[i][0]} {frame[i][1]}\n")
            print("Sending Frame", frame[i][0], "...", frame[i][1])
print("\nWaiting for ACK...")
time.sleep(2)

try:
    with open("Receiver_Buffer.txt", "r") as file:
        ack = int(file.read().strip())
    print("ACK received:", ack)
except:
    print("no ack received")
    continue

if ack == end-1:
    print("ACK is correct")
    start = ack

else:
    print("NACK / Wrong ACK")
    print("Resending frame from frame", ack+1)
    if ack >= start:
        start = ack+1
    else:
        start = start

print("\nAll frames transmitted successfully")
```

Receiver.py

```
import time
import os
print("Receiver started...")
expected_frame = 0
old_buffer = ""
while True:
    if not os.path.exists("sender_Buffer.txt"):
        time.sleep(1)
        continue
    with open("sender_Buffer.txt", "r") as file:
        data = file.read()
    if data == old_buffer:
        time.sleep(1)
        continue
    old_buffer = data
    lines = data.strip().split("\n")
    print("\nReceived Frames:")
    last_ack = expected_frame - 1
```

```
for line in lines:
    parts = line.split(" ", 1)
    frame_no = int(parts[0])
    frame_data = parts[1]
    print("Frame", frame_no, ":", frame_data)
    if frame_no == expected_frame:
        print("Frame accepted")
        last_ack = frame_no
        expected_frame += 1
    else:
        print("Wrong Frame")
        break
print("Sending ACK:", last_ack)
with open("Receiver_Buffer.txt", "w") as file:
    file.write(str(last_ack))
last_frame = int(lines[-1].split(" ", 1)[0])
if last_ack == last_frame:
    print("\n All frame received successfully!")
    break
time.sleep(1)
```

Output:

```
> python sender.py
Enter window size: 3
Enta the message: computer networks
Frames Created:
Frame No: 0  Data: Compu
Frame No: 1  Data: ter N
Frame No: 2  Data: etwork
Frame No: 3  Data: ks
Sending frames...
Sending frame 0: Compu
Sending frame 1: ter N
Sending frame 2: etwork
Waiting for ACK
ACK received: 3
NACK / Wrong ACK!
Resending frame from frame 3
```

> Python Receiver.py

```
Receiver started...
Receive frames:
Frame 0: Compu
Frame accepted
Frame 1: ter N
Frame accepted
Frame 2: etwor
Frame accepted
Sending ACK: 2
All frames received successfully!
```

Result: The sliding window protocol was successfully implemented using python frames were transmitted and acknowledged successfully, demonstrating reliable data transmission.

Experiment 8

NMAP Host discovery

AIM: To discover live hosts using nmap scans (ARP, ICMP, TCP) in tryHackme platform

Introduction

Nmap is a free and open source scanning tool used to discover live host, and identify the services running with version

NMap host discovery using ARP :-

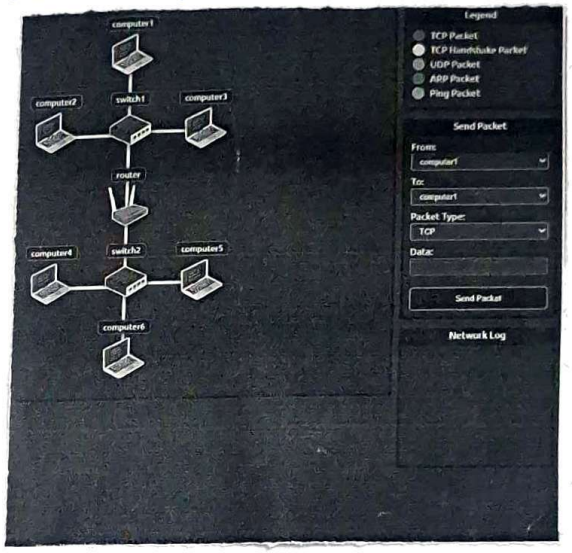
ARP is used to discover hosts on the same local subnet nmap sends ARP request and live hosts respond with ARP replies. The PR is used for ARP based host discovery.

Nmap host discovery using ICMP:

ICMP provides different req/reply mechanics for discovery live hosts Nmap suggest ICMP Echo (PE), Timestamp (PP) and address mask (PM) ones.

Nmap discovery using TCP and UDP :-

When ICMP is filtered, Nmap can use TCP and UDP packets to determine whether a host is online. TCP host discovery includes TCP SYN ping (-PS) and TCP ACK ping (-PA), while UDP host discovery use -PU



- From **computer4**
- To **computer4** (to indicate it is broadcast)
- Packet Type: "ARP Request"
- Data: **computer6** (because we are asking for **computer6's** MAC address using ARP Request)

How many devices can see the ARP Request?

4

✓ Correct Answer

Did computer6 reply to the ARP Request? (yea/nay)

yea

✓ Correct Answer

Result: Thus, live hosts in the network were successfully discovered using nmap host discovery techniques such as ARP, ICMP and TCP, UDP ping scans. The different scanning methods are used to identify active hosts.

Experiment - 9

Subnetting

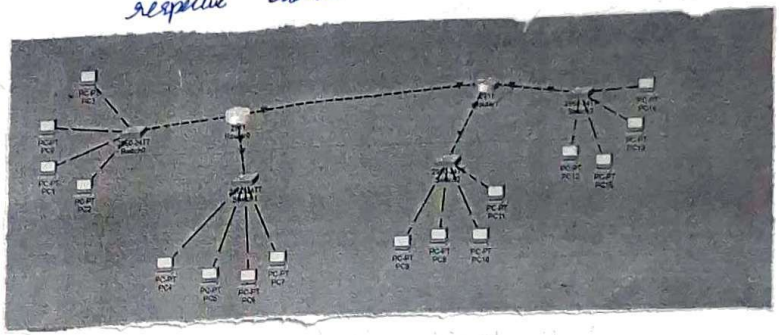
AIM:- Implementation of subnetting in CISO packet tracer simulator.

Introduction:-

Subnetting is the process of dividing a large IP network into smaller logical networks called subnets. Classless subnetting allows us to use subnet masks other than default class-based masks.

Procedure:

1) Open cisco packet tracer
2) Add required routers, switches and PCs
3) Connect the devices using appropriate cables.
4) Configure the routers with the required IP address.
5) Configure the router interface using the CLI:-
   - enable
   - configure terminal
   - select the required Fast ethernet;
   - Assign the IP address and subnet mask.
   - Use no shutdown to activate the interface.
6) Configure the switch ports in access mode where required
7) Assign static IP address, subnet mask and default gateways to the PC
8) Use the 124 subnet mask: 255.255.255.224.
9) Configure the four subnets:-
   - 192.168.1.0/27
   - 192.168.2.0/27
   - 192.168.3.0/27
   - 192.168.4.0/27
10) Verify the router interface and PCs belong to their respective router.



Result. Thus, subnetting was successfully implemented using CISCO packet tracer.

Experiment 10 (a)

Interworking with routers

AIM:-
Interworking with routers in CISCO packet tracer simulator.

Procedure:-

Step 1: Configuring Router 1

1) Open cisco packet tracer and place one router and 2 PCs
2) Select the router and open CLI and press enter to access the ~~comm~~ cmd.
3) Enter privileged mode using enable.
4) Enter global configuration mode using config terminal
5) Configure Fast Ethernet 0/0 with
   IP address: 192.168.20.1
   Subnet mask: 255.255.255.0
6) Use no shutdown to activate the interface

Step 2: Configuring PCs

Device	IP address	Subnet Mask	Default Gateway
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1

Step 3:- Connecting PC's with Router.

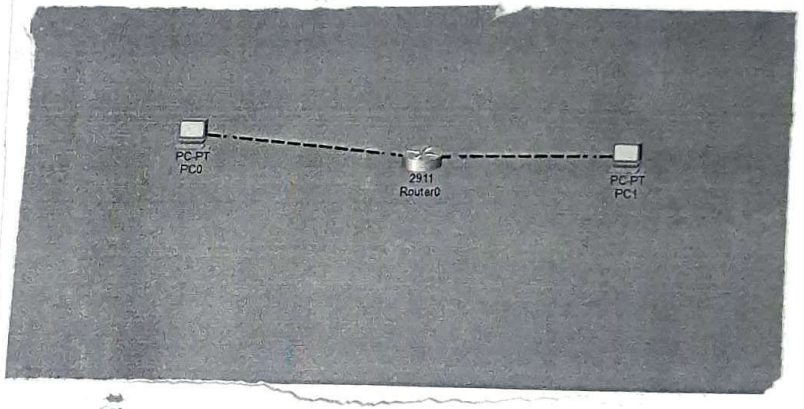
1) Connect PC0 Fast Ethernet to Router 1 Fast Ethernet 0/0 using a copper straight through cable.
2) Connect PC1 fast ethernet 0 to Router 1 fast ethernet 0/1 using a copper straight through cable.
3) Wait until the interface become active.

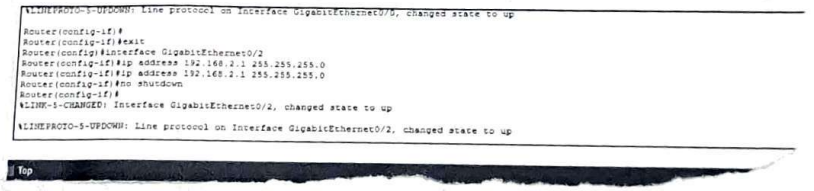
5) Verify that PDV reaches PC1 and is received back to PC0

Configuration Table:

Router configuration

Device	Fast ethernet 0/0	Subnet mask	Fastethernet 0/1
Router 1	192.168.10.0	255.255.255.0	192.168.20.1





Result: Thus, a single internetwork was successfully designed and configured using a router in cisco packet tracer.

Experiment no (6)

Internetwork using wired router

AIM: To design and configure an internetwork using a wireless router, DHCP and internet cloud in cisco packet tracer.

Procedure:-

Step 1: Build the network topology.

1) Open cisco packet tracer and add the devices like PC, router, modem internet cloud and server.
2) Connect the devices using the appropriate cable

Step 2: Configure the Router.

1) Wireless Router → GUI → Wireless.
2) Go to setup tab and ensure DHCP is enabled
3) Configure the server as 208.67.220.220
4) Save the setting

Step 3: Configure the laptop.

1) Laptop → Physical tab → power off → Remove ethernet cable and add WPC300N wireless module → power on the Laptop.
2) Desktop → PC. select connect tab and select hardware
3) Connect the laptop to wireless network.

Step 4: Configure the PC

1) PC → Desktop → IP config
2) The PC automatically receives an IP address from the router.
3) Use the command prompt and ipconfig/all to verify the configuration.

~~Step 5 : Config~~

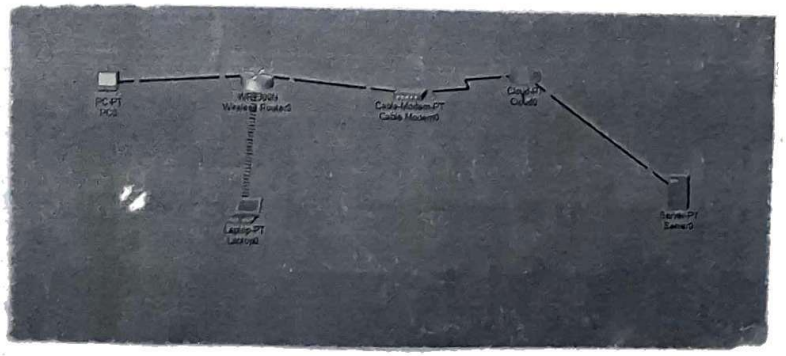
Step 6: Configure the cisco.com server DHCP configuration.

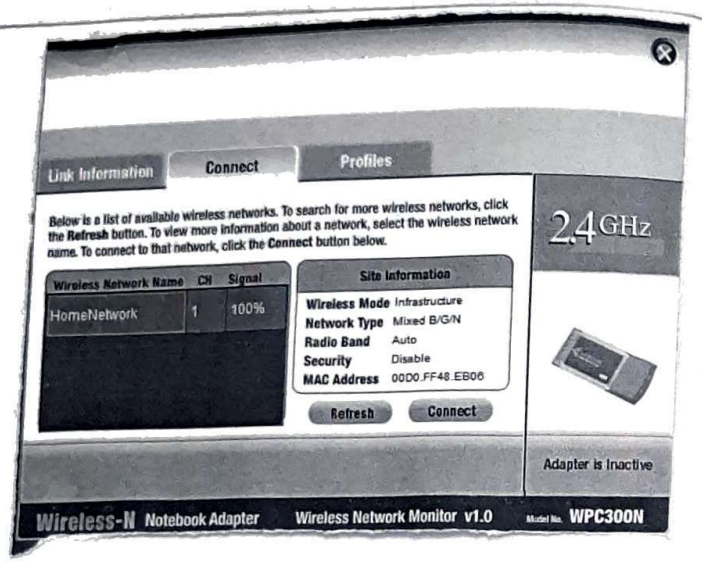
Parameter	Value
PoolName	DHCP pool
Default Gateway	208.67.220.220
DNS server	208.67.220.220
Starting IP	208.67.220.1
Subnet Mask	255.255.255.0
Maximum use	50

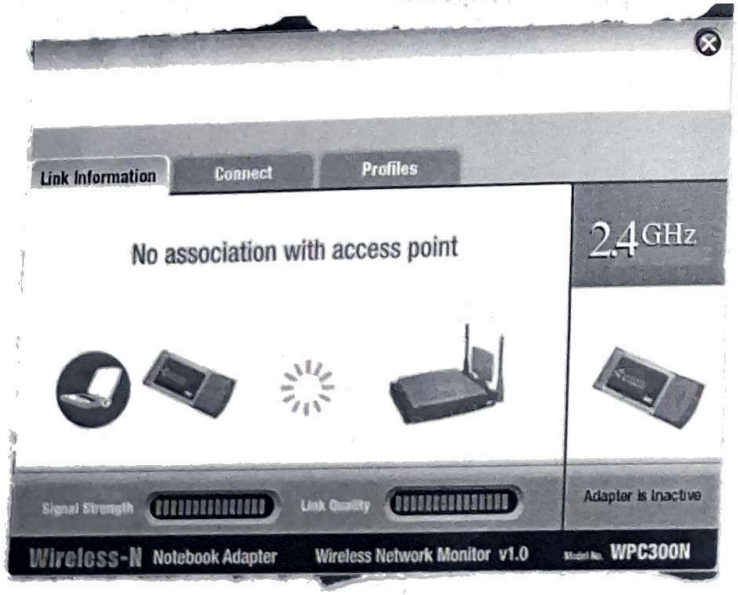
DNS configuration :-

Parameter	Value
Name	cisco.com
Type	Record
Address	208.67.220.220

Step 7:- Verify connectivity.







Result: Thus the wireless router, DHCP server and internet cloud were successfully configured in cisco packet tracer and connectivity was verified successfully

Experiment VI

Static routing

AIM: To simulate and configure static routing with primary and backup route using cisco packet tracer and verify connectivity between different networks.

Procedure :-

1) Open cisco packet tracer and create the given topology with Router 0, Router 1, Router 2 and PCs.
2) Configure the IP address on all the router interfaces and PC's according to the given topology.
3) Open the CLI of Router 0 and enter configuration mode.
4) Config a separate host route 30.0.0.100/32 with router 2 as the primary route and router as backup route.
5) On router 1, configure static route for network 10.0.0.0/8 and 40.0.0/8 with primary and backup route.
6) On router 2, config static route for network 10.0.0.0/8 and 30.0.0.0/8.
7) Use the command: show ip route static to verify the configured static route.
8) Static route can be removed using: no ip route.
9) Verify the routing table again using show ip route static

Router 1

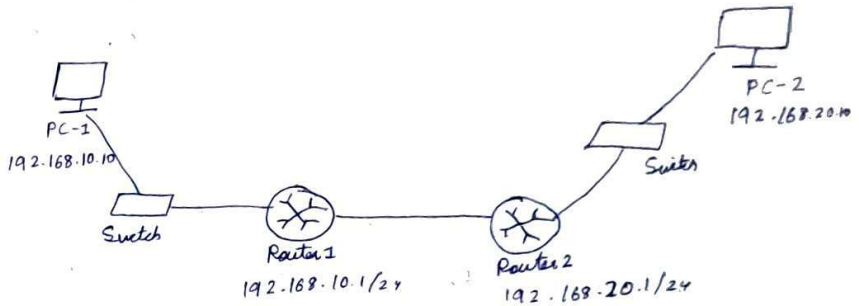
Router1 > enable
Router 1# configure terminal
Router 1 (config)# interface GigabitEthernet 0/0
Router 1 (config-if)# ip address 192.168.10.1 255.255.255.0
Router 1 (config-if)# no shutdown
Router 1 (config-if)# exit

Router 1 ~~enable~~ (config)# interface GigabitEthernet 0/1
Router 1 (config-if)# ip address 10.0.01 255.25.255.75 2
Router 1 (config-if)# no shutdown
Router 1 (config-if)# exit

Router 2

Router 2 > enable
Router 2# configure terminal
Router 2 (config-if)# interface GigabitEthernet 0/0
Router 2 (config-if)# ip address 192.168.20.1 255.255.255.0
Router 2 (config-if)# no shutdown
Router 2 (config-if)# exit

Router 2 (config)# interface GigabitEthernet 0/1
Router 2 (config-if)# ip address 10.0.0.2 255.255.255.28
Router 2 (config-if)# no shutdown
Router 2 (config-if)# exit



Result: Thus, the routing with passing and backup routes was successfully configured and simulated using CISCO.

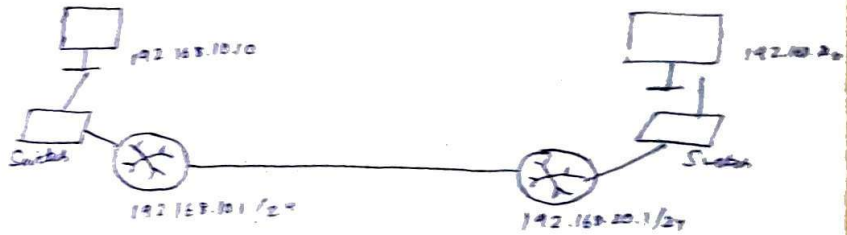
Experiment : 8

RIP (Routing Information Protocol)

AIM :- To simulate and configure RIP (routing information protocol) using cisco packet tracer and verify dynamic routing between different networks

Procedure

1) Open cisco packet tracer and create the given topology with router 0, Router 1, Router 2, PC0 and PC1
2) Assign IP address to PCs and router interface according to table
3) Use no shutdown to activate all router interface
4) For DCE serial interface, configure the required clock rate and bandwidth.
5) Configure RIP on R0 using :-
   router rip
   network 10.0.0.0
   network 192.168.1.252
6) Configure RIP on R1 using :-
   router rip
   network 192.168.1.252
   network 192.168.1.248
7) Configure RIP on R2 using :-
   router rip
   network 20.0.0.0
   network 192.168.1.250
   network 192.168.1.248
8) Allow RIP to exchange routing information automatically between the router
9) Use traceroute cmd to observe the path followed by packet between the network



Router 1

```
Router 1 (config) # router rip
Router 1 (config-router)# version 2
Router 1 (config-router)# no auto-summary
Router 1 (config-router)# network 192.168.10.0
Router 1 (config-router)# network 10.0.0.0
Router 1 (config-router)# passive-interface g0/0
```

Router 2

```
Router 2(config)# router rip
Router 2(config-router)# version 2
Router 2(config-router)# no auto-summary
Router 2(config-router)# network 192.168.20.0
Router 2(config-router)# network 10.0.0.0
Router 2(config-router)# passive-interface g0/0
```

Result :- Thus RIP was successfully configured and simulated using CISCO

EXP

End - End Communication at transport layer

AIM:- To implement an end to end communication chat program at transport layer using TCP sockets and UDP sockets.

CODE:-

TCP

```
import socket
import threading

def start_server():
    server = socket.socket(socket.AF_INET,
                           socket.SOCK_STREAM)
    server.bind(('127.0.0.1', 5000))
    server.listen(1)
    print("[SERVER] Listening at 127.0.0.1:5000")
    conn, addr = server.accept()
    print(f"[SERVER] Connected by {addr}")
    while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() == 'quit':
            print("[SERVER] Connection closed by client")
            break
        print(f"[SERVER] Received {data}")
        conn.send(f"Server received: {data}".encode())
    conn.close()
    server.close()

def start_client():
    client = socket.socket(socket.AF_INET,
                           socket.SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
    print("[Client] Connected to Server")
    while True:
        msg = input("Enter: ")
        client.send(msg.encode())
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
        print(f"[SERVER Response] {response}")
    client.close()

if __name__ == "__main__":
    server_thread = threading.Thread(target=start_server,
                                     daemon=True)
    server_thread.start()
    start_client()
```

Output:-

```
[SERVER] Listening on 127.0.0.1:5000
[Client] Connected to server
[SERVER] Connected by ('127.0.0.1', 49728)
Enter: ok
[SERVER] Received: ok
[SERVER Response] Server received: ok
```

UDP

```
import socket
import threading

def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_
                      DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")
    while True:
```

```
            data, addr = s.recvfrom(1024)
            msg = data.decode()
            if msg.lower() == 'quit':
                print(f"[SERVER] Client {addr} ended")
                break
            s.sendto(data, addr)

        s.close()
        print("[Server] Stopped.")


def udp_client():
    c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message: ")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[Client] Closing...")
            break
        data, _ = c.recvfrom(1024)
        print("Echo from Server:", data.decode())

    c.close()
    print("[Client] Stopped.")

if __name__ == "__main__":
    threading.Thread(target=udp_server, daemon=True).start()
    udp_client()
```

Output

```
[SERVER] Running on 127.0.0.1:12000
Enter message: hi
[SERVER] Received from ('127.0.0.1', 53215): hi
Echo from server: hi
```

Result: The UDP and TCP echo program was implemented and executed successfully.

Ping Program

AIM: To write and execute a python program to implement a custom ping

CODE

```
import os
import platform
import subprocess

def ping (host = "google.com", count = 4):
    param = "-n" if platform.system().lower()==
                "windows" else "-c"
    cmd = ["ping", param, str(count), host]
    try:
        output = subprocess.check_output(cmd, universal_
                                         newlines=True)
        print(output)
    except Exception as e:
        print(f"Ping failed {e}")

if __name__ == "__main__":
    ping("8.8.8.8")
```

Output

```
Pinging 8.8.8.8 with 32 byte of data
Reply from 8.8.8.8 bytes = 32 time = 60ms TTL=113
Reply from 8.8.8.8 byte = 32 time = 36 ms TTL=113

Ping statistics for 8.8.8.8
    Packets : Sent=4, Received=4
Approx round Trip
    Minimum =33ms, Maximum=60ms, Average=40ms
```

Result: Thus the ping program was executed successfully

Experiment 14

Packet sniffing

AIM: To write a python program using sockets / Scapy to implement network packet sniffing.

CODE:-

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP
def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst

        if protocol == 1:
            protocol_name = "ICMP"
        elif protocol == 6:
            protocol_name = "TCP"
        elif protocol == 17:
            protocol_name = "UDP"
        else:
            protocol_name = "Unknown Protocol"
        print(f"Protocol: {protocol_name}")
        print(f"Source IP: {src_ip}")
        print(f"Destination IP: {dst_ip}")
        print("-" * 50)

sniff(iface = 'Wifi', prn = packet_callback, filter = "ip", store=0)
```

Output

Protocol: TCP
Source IP : 57.153.141.117
Destination IP : 192.168.1.6

Protocol : TCP
Source IP : 192.168.1.6
Destination IP : 57.153.141.117

Protocol : TCP
Source IP : 192.168.1.6
Destination IP : 57.153.141.117

Protocol : TCP
Source IP : 192.168.1.1
Destination IP : 192.168.1.6

Result: The packet sniffing program using Scapy was successfully implemented and executed.